

The Art of R Programming

A Tour of Statistical Software Design

by Norman Matloff

errata updated to print 14

Page	Error	Correction	Print corrected
1	If not, see Appendix A. for installation instructions.	If not, see Appendix A for installation instructions.	Print 3
51	<pre>vud <- diff(d)</pre>	<pre>vud <- diff(v)</pre>	Print 3
53	<pre>for (gen in c("M","F")) grps[[gen]] <- which(aba==gen)</pre>	<pre>for (gen in c("M","F")) grps[[gen]] <- which(aba[,1]==gen)</pre>	Print 2
65	<pre>randomnoise <- matrix(nrow=lrows, ncol=ncols, runif(lrows*lcols)) newimg@grey <- (1-q) * img@grey + q * randomnoise</pre>	<pre>randomnoise <- matrix(nrow=lrows, ncol=lcols, runif(lrows*lcols)) newimg@grey[rows,cols] <- (1-q) * img@grey[rows,cols] + q * randomnoise</pre>	Print 4
66	<pre>newimg@grey <- (1-q) * img@grey + q * randomnoise</pre>	<pre>newimg@grey[rows,cols] <- (1-q) * img@grey[rows,cols] + q * randomnoise</pre>	Print 4
67	<pre>> z <- c(5,12,13) > x[z %% 2 == 1,] [,1] [,2]</pre>	<pre>> x[z %% 2 == 1,] [,1] [,2]</pre>	Print 3
68	<pre>[1,] 1 4 [2,] 3 6</pre>	<pre>[1,] 1 2 [2,] 3 4</pre>	Print 3
77	Recall that due to the symmetry of the matrix, we skip the early part of each row, as is seen in the expression (i+1):(1x-1) in line 18. But that means that the call to which.min() in that line will return the minimum's index <i>relative</i> to the range (i+1):(1x-1).	Recall that due to the symmetry of the matrix, we skip the early part of each row, as is seen in the expression (i+1):(1x-1) in line 18. But that means that the call to which.min() in that line will return the minimum's index <i>relative</i> to the range (i+1):(1x-1).	Print 3

Page	Error	Correction	Print corrected
93	<pre>> nwords <- length(ssnyt) > barplot(freqs9)</pre>	<pre>> nwords <- length(ssnyt) > freqs9 <- sapply(ssnyt[round(0.9*nwords):nwords],length) > barplot(freqs9)</pre>	Print 2
116	<pre>shang3</pre>	<pre>shang4</pre>	Print 2
128	<pre>fl.1 a bc 5 2 1 12 1 1 13 1 0</pre>	<pre>fl.1 a bc 5 2 0 12 1 1 13 2 1</pre>	Print 2
130	<pre>> ctt/5</pre>	<pre>> cttab/5</pre>	Print 2
131	<pre>> apply(ctt,1,sum)</pre>	<pre>> apply(cttab,1,sum)</pre>	Print 2
133	<pre>f(argslist[[1],argslist[[2]],...)</pre>	<pre>f(argslist[[1]],argslist[[2]],...)</pre>	Print 2
137	This says that z[1], 0.88114802, fell into bin 9, which was (0,0,0.1]; z[2], 0.28532689, fell into bin 3; and so on.	This says that z[1], 0.88114802, fell into bin 9, which was (0.8,0.9]; z[2], 0.28532689, fell into bin 3; and so on.	Print 2
148	Good software design, however, should be mean that you can glance through a function's code ...	Good software design, however, should mean that you can glance through a function's code ...	Print 3
151	<pre>> f(3,2) [1] 1 > g <- function(h,a,b) h(a,b) > g(f1,3,2) [1] 5 > g(f2,3,2) [1] 1</pre>	<pre>> f(3,2) [1] 1 > g <- function(x) x^2 > body(g) <- quote(2*x+3) > g function (x) 2 * x + 3 > g(8) [1] 19</pre>	Print 5

Page	Error	Correction	Print corrected
151	<pre>> g <- function(h,a,b) h(a,b) > body(g) <- quote(2*x + 3) > g function (x) 2 * x + 3 > g(3) [1] 9</pre>	<pre>> g <- function(h,a,b) h(a,b) > body(g) <- quote(2*x + 3) > g function (x) 2 * x + 3 > x <- 3 > g(3) [1] 9</pre>	Print 2
155	<pre>> f(2) [1] 88</pre>	<pre>> f(2) [1] 112</pre>	Print 2
160	<pre>> oddsevens function(v){ odds <- which(v %% 2 == 1) evens <- which(v %% 2 == 1) list(o=odds,e=evens) }</pre>	<pre>> oddsevens function(v){ odds <- which(v %% 2 == 1) evens <- which(v %% 2 == 0) list(o=odds,e=evens) }</pre>	Print 2
163	<pre>makecorpdfs(c("MICROSOFT CORPORATION","ms","INTEL CORPORATION","intel"," SUN MICROSYSTEMS, INC.","sun","GOOGLE INC.","google"))</pre>	<pre>makecorpdfs(c("MICROSOFT CORPORATION","ms","INTEL CORPORATION","intel"," SUN MICROSYSTEMS, INC.","sun","GOOGLE INC.","google"))</pre>	Print 2
164	... when we discuss appropriate use global variables in the next section.	... when we discuss appropriate use of global variables in the next section.	Print 3
176	3. Within f(), piece together the results of (b) to solve the original problem.	3. Within f(), piece together the results of (2) to solve the original problem.	Print 3
178	... while the right subtree stores the elements that are larger than the value in this mode .	... while the right subtree stores the elements that are larger than the value in this node .	Print 3

Page	Error	Correction	Print corrected
185	<pre> 26 \end{Code} 27 28 Let's test it. 29 30 \begin{Code} 31 > b <- newbookvec(c(3,4,5,5,12,13)) 32 > b 33 \$vec 34 [1] 3 4 5 5 12 13 35 36 \$wrts 37 [1] 0 0 0 0 0 0 38 39 attr(,"class") 40 [1] "bookvec" 41 > b[2] 42 [1] 4 43 > b[2] <- 88 # try writing 44 > b[2] # worked? 45 [1] 88 46 > b\$wrts # write count incremented? 47 [1] 0 1 0 0 0 0 </pre>	Let's test it. <pre> > b <- newbookvec(c(3,4,5,5,12,13)) > b \$vec [1] 3 4 5 5 12 13 \$wrts [1] 0 0 0 0 0 0 attr(,"class") [1] "bookvec" > b[2] [1] 4 > b[2] <- 88 # try writing > b[2] # worked? [1] 88 > b\$wrts # write count incremented? [1] 0 1 0 0 0 0 </pre>	Print 3
191	The expression <code>notp[-i]</code> computes the product of all the elements of <code>notp</code> , ...	The expression <code>prod(notp[-i])</code> computes the product of all the elements of <code>notp</code> , ...	Print 3
194	For instance, to find our more about the chi-square function for quantiles, ...	For instance, to find out more about the chi-square function for quantiles, ...	Print 3
197	<pre> > a <- matrix(c(1,1,-1,1),nrow=2,ncol=2) > b <- c(2,4) > solve(a,b) [1] 3 1 > solve(a) [,1] [,2] [1,] 0.5 0.5 [2,] -0.5 0.5 </pre>	<pre> > a <- matrix(c(1,-1,1,1),nrow=2) > b <- c(2,4) > solve(a,b) [1] -1 3 > solve(a) [,1] [,2] [1,] 0.5 -0.5 [2,] 0.5 0.5 </pre>	Print 3
206	Recalling that R lists are often used to store several related variables in one basket, we se up a list <code>comdat</code> .	Recalling that R lists are often used to store several related variables in one basket, we set up a list <code>comdat</code> .	Print 3
228	<pre> > save(hz,"hzfile") </pre>	<pre> > save(hz,file="hzfile") </pre>	Print 3

Page	Error	Correction	Print corrected
264	On a Mac, call <code>macintosh()</code>.	On a Mac, call <code>quartz()</code>.	Print 3
276	<pre>g <- function(t) { return (t^2+1)^0.5 } # define g()</pre>	<pre>g <- function(t) { return ((t^2+1)^0.5) } # define g()</pre>	Print 3
295	<pre>returns the minimum value of d[i,j], i != j, and the row/col attaining that minimum, for square symmetric matrix d; no special policy on ties; motivated by distance matrices</pre>	<pre># returns the minimum value of d[i,j], i != j, and the row/col attaining # that minimum, for square symmetric matrix d; no special policy on # ties; # motivated by distance matrices</pre>	Print 3
345	As of this writing, GPU has not yet become common among R users.	As of this writing, GPU <code>programming</code> has not yet become common among R users.	Print 3